Wrocław
University
of Science
and Technology

Software Engineering

Requirements Engineering: agile approach.

Marcin Jodłowiec

October 29th, 2025

Wrocław University of Science and Technology

Agenda

- What is Agile?
  - The Agile Manifesto
- Agile vs waterfall
- Agile requirements fundamentals: lean thinking
- Agile requirements: Team level
  - User Stories
  - The Backlog
  - Tasks
  - Spikes
- Agile requirements: Program level
  - Vision
  - Features
  - Roadmap and release planning
- Agile requirements: Portfolio level
  - Investment themes
  - Epics
- Summary
- The metamodel

Wrocław
University
of Science
and Technology

What is Agile?

What is Agile?

What is Agile?

Agile is an **iterative and adaptive approach** to software development that emphasizes **people, collaboration, and responsiveness to change** over rigid planning and documentation.

Agile $\neq$ "lack of plan" $\rightarrow$ it means adapting the plan when reality changes.

Essence of agile (Agile Manifesto, 2001)

- Short, time-boxed **iterations** (sprints, increments)
- Continuous **feedback** from stakeholders
- **Evolving requirements** throughout the project
- **Self-organizing teams** delivering value

The Agile Manifesto

https://agilemanifesto.org/

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions	over	processes and tools
Working software	over	comprehensive documentation
Customer collaboration	over	contract negotiation
Responding to change	over	following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Wrocław
University
of Science
and Technology

Agile vs waterfall

Agile vs waterfall

The development process

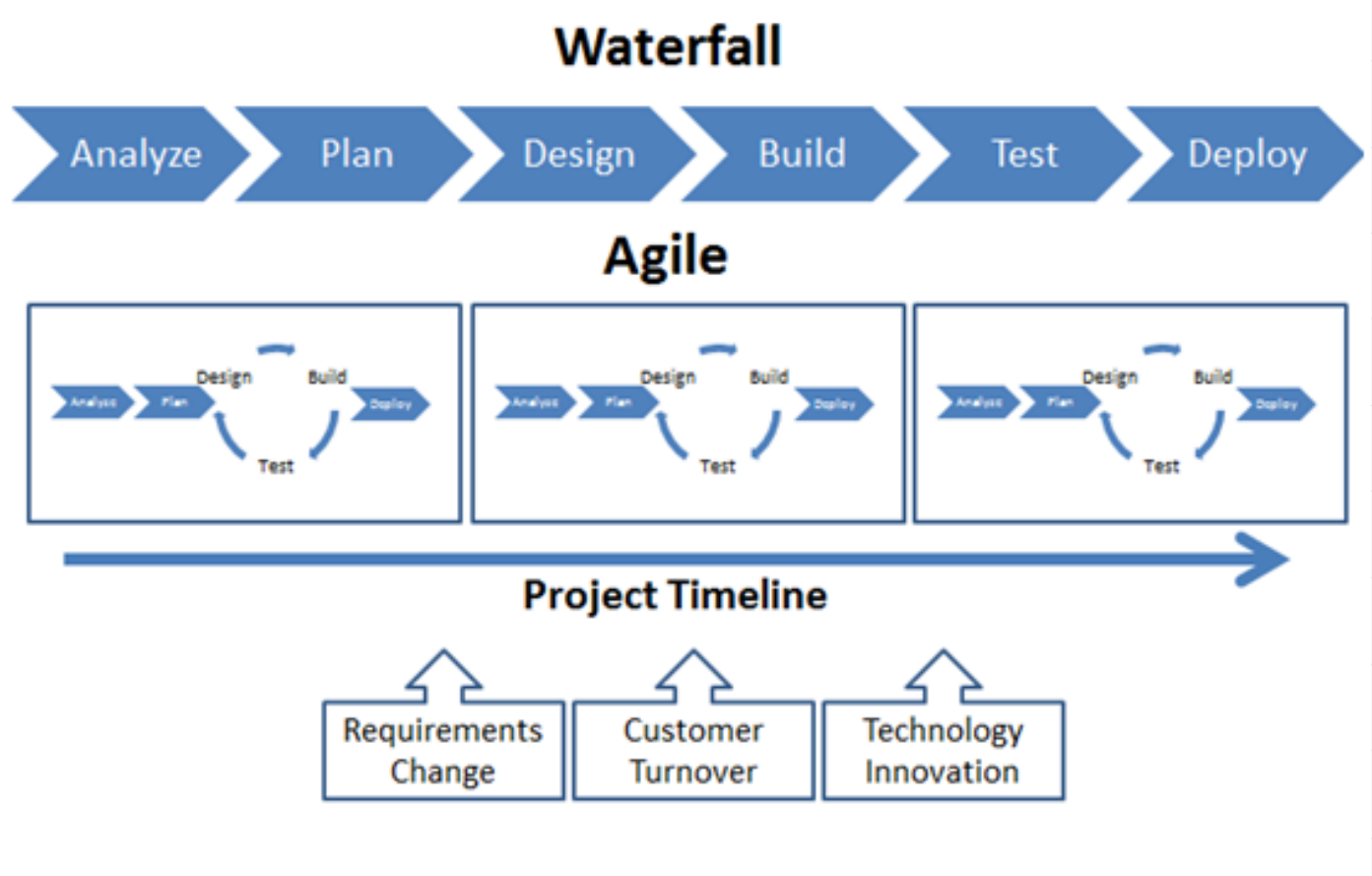


Figure: Agile vs Waterfall development process

Source: http://www.neoglobeconsulting.com/blog/2014/3/23/why-go-agile-understanding-the-benefits-of-the-agile-method-of-development

Agile vs waterfall
ROI

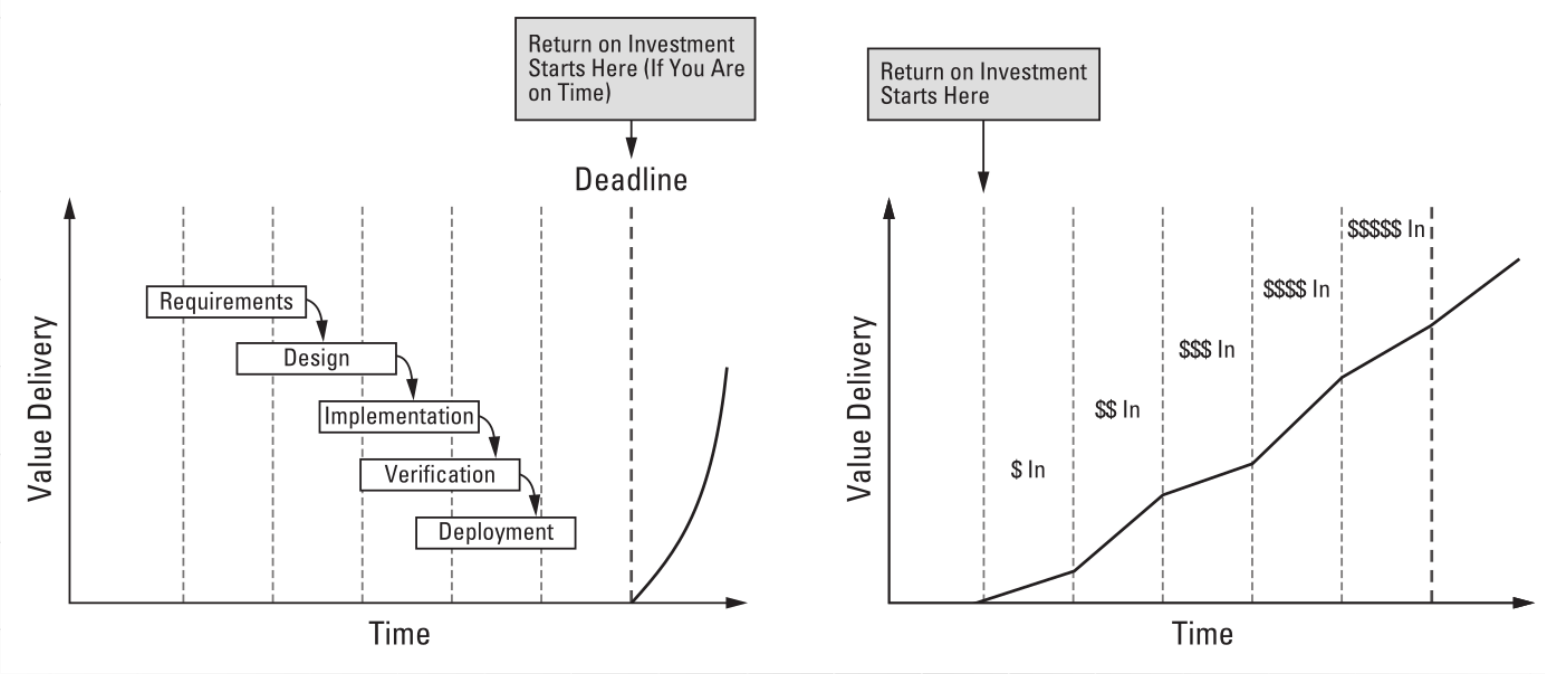


Figure: Return on investment in agile vs waterfall projects[1]

[1] Leffingwell 2011.

Agile vs waterfall

The iron triangle

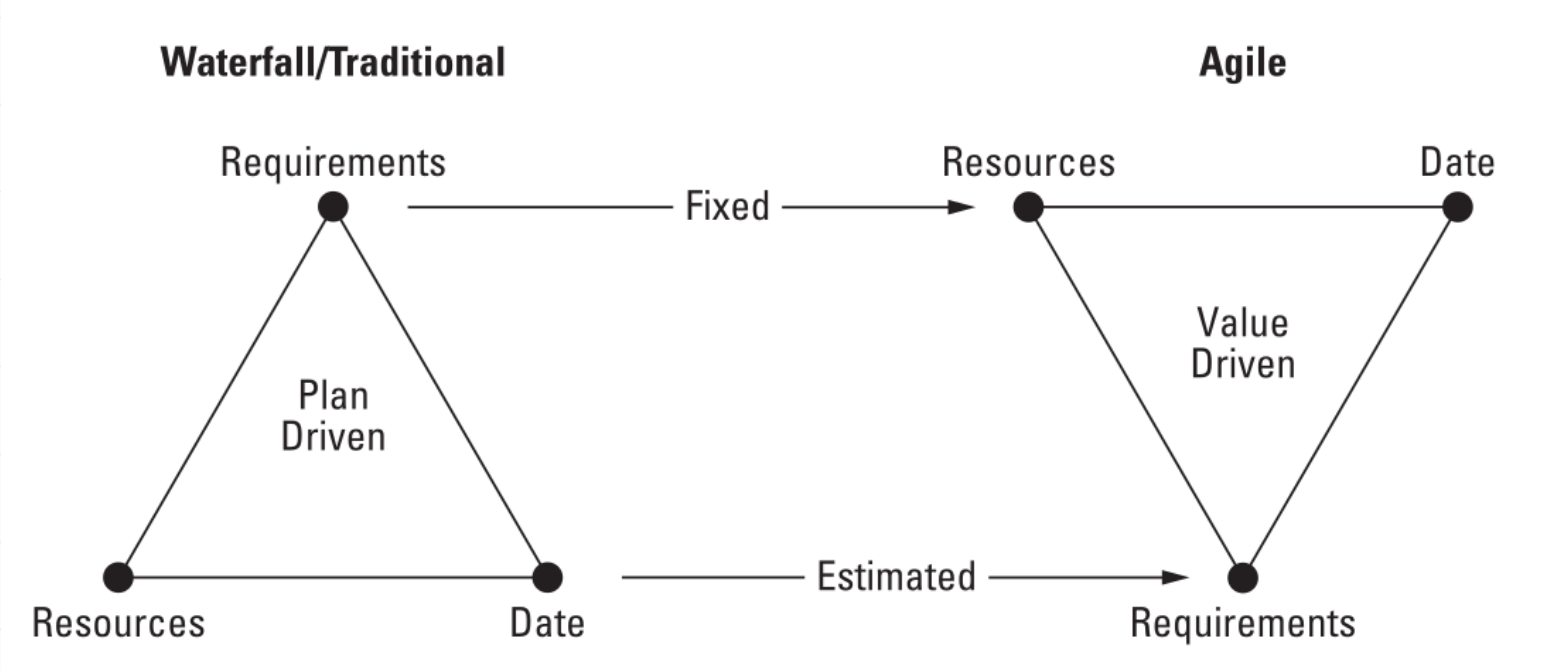


Figure: The iron triangle in agile vs waterfall approach[2]

[2] Leffingwell 2011.

Agile requirements fundamentals: lean thinking

Lean Philosophy and the Deming Influence I

Lean Philosophy:

- A mindset focused on **continuous improvement**, **waste elimination**, and **value creation**.
- Sees organizations as **systems** that must be optimized as a whole, not locally.
- Emphasizes **learning**, **respect for people**, and **empowered teams**.

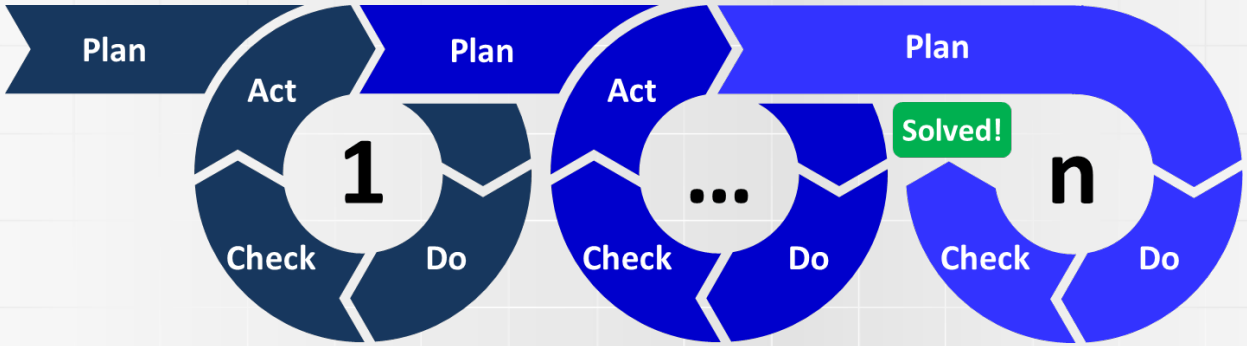
Deming's Influence:

- W. Edwards Deming introduced the idea that **quality is built into the process**.
- His **PDCA Cycle (Plan–Do–Check–Act)** became the backbone of continuous improvement (*Kaizen*).

PDCA – Continuous Improvement Loop:

1. **Plan** – Identify a goal or problem and design an experiment or change.
2. **Do** – Implement the plan on a small scale.
3. **Check** – Measure results and compare with expectations.
4. **Act** – Standardize successful practices or adjust and try again.

Lean Philosophy and the Deming Influence II



Lean Philosophy and Agile:

- Iteration and feedback cycles reflect the PDCA loop.
- Teams continuously refine processes, requirements, and product value.
- Lean thinking inspires Agile teams to eliminate waste and go *kaizen* (from Japanese: kai (改) zen (善), "change for better").

Hierarchy of Requirements in Agile Development

Why hierarchy?

- Agile organizations operate at multiple levels: enterprise, program, and team.
- Requirements must be **aligned across levels** to connect strategy with execution.
- Each level refines and decomposes requirements from the level above.

Core ideas:

- High-level goals evolve into smaller, testable user stories.
- Feedback at every level drives continuous adaptation.
- Lean principles guide prioritization and decision-making.

Agile requirements: Team level

The Agile Team I

Agile team

Agile Teams are small, cross-functional groups (typically 3–9 people) that define, build, test, and deliver software value to the user.

Core Roles:

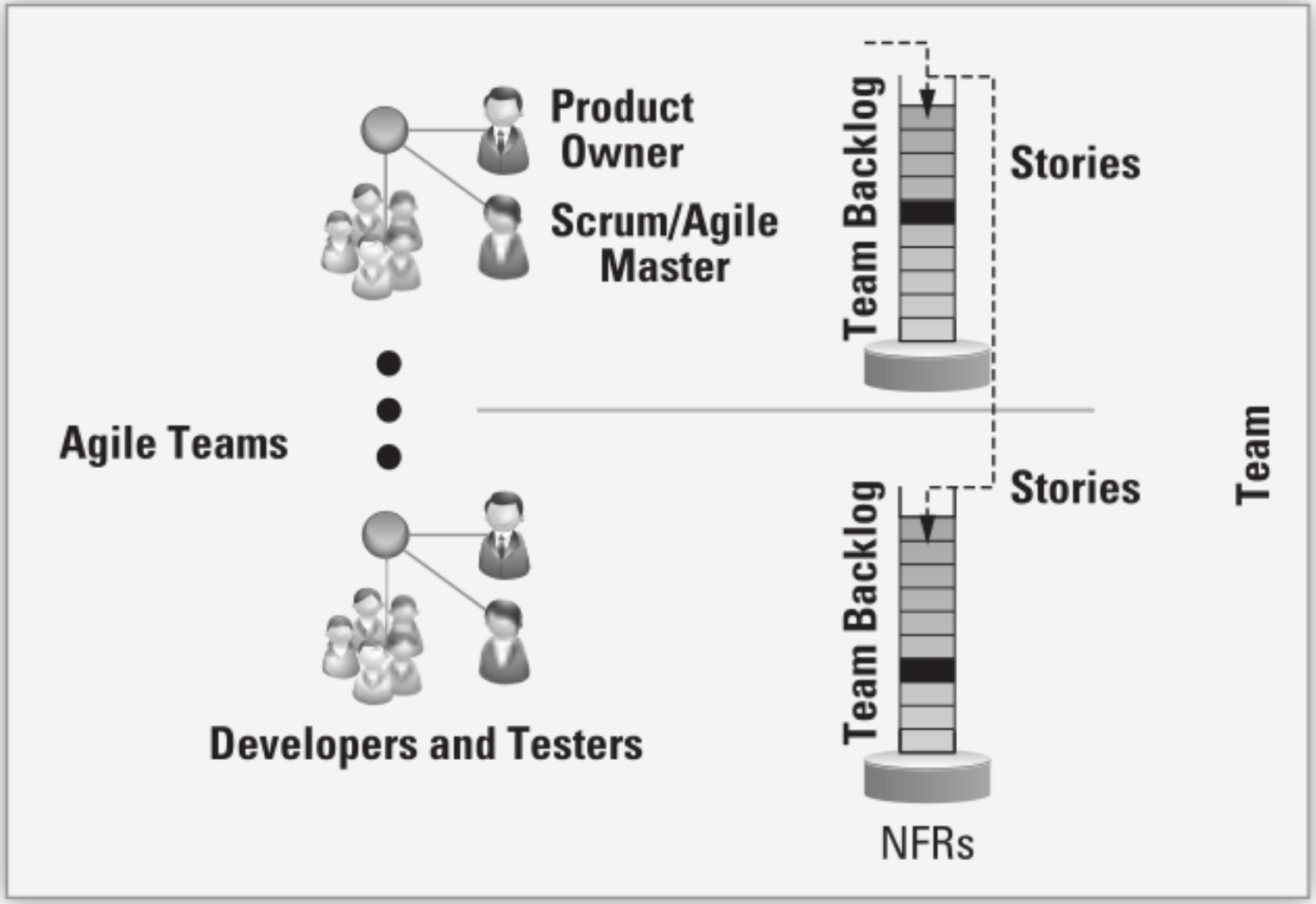
- **Product Owner** – defines and prioritizes the *team backlog*, represents customer needs, and ensures continuous collaboration.
- **Scrum Master/Agile Coach** – facilitates the process, removes impediments, and fosters continuous improvement and team performance.
- **Developers and Testers** – implement, integrate, and test user stories; jointly responsible for delivering working, tested software.

The Agile Team II

Team Characteristics:

- Self-organizing and cross-functional.
- Owns end-to-end responsibility for quality and delivery.
- Collaborates daily, following Agile Manifesto principle #4: *"Business people and developers must work together daily."*
- Often organized as **feature teams** (user-facing value) or **component teams** (shared infrastructure).

The Agile Team III



User Stories

User Stories: the Agile form of Requirements

- User stories often replace or or domplement detailed SRS-style traditional **software requirements** in agile environments.
- They are **brief statements of intent** that describe what the system should do for a specific user.
- Typical format:

  *As a <**user role**>, I can <**activity**> so that <**business value**>.*

  Example: *As a **registered user**, I can **reset my password** so that **I can regain access to my account if I forget it**.*
- Encourage focus on user value and business outcomes, not on technical details.

The Backlog

- Contains all **user stories**, plus defects, refactors, and infrastructure work.
- Maintained and prioritized by the **Product Owner**.
- Represents the team's **pipeline of value delivery**.
- Activities: identifying, prioritizing, elaborating, implementing, testing, and accepting stories.

The Backlog

Example

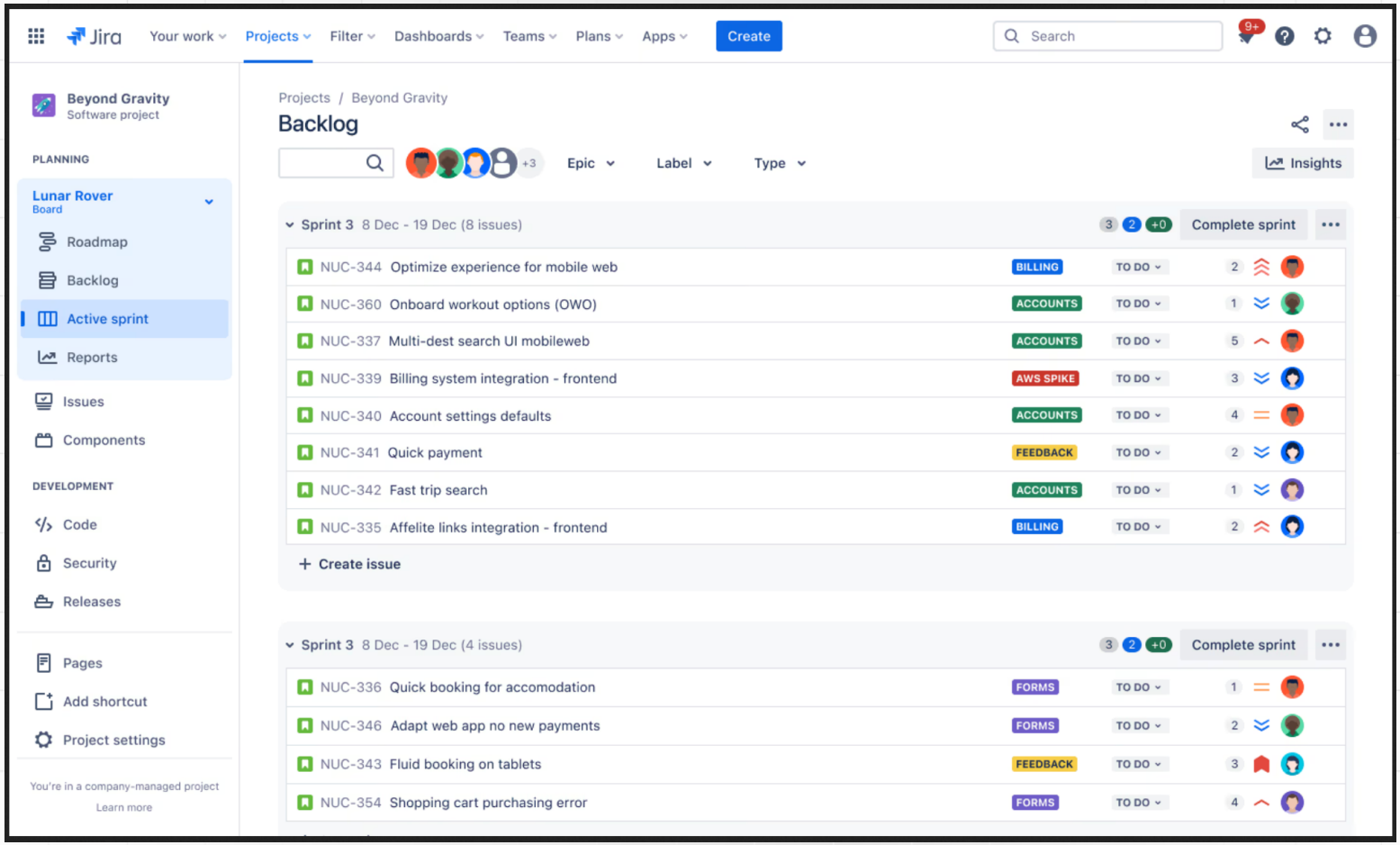


Figure: The Agile Team Backlog

Source: https://productschool.com/blog/product-fundamentals/product-backlog-examples

Backlog vs Kanban Board

Two complementary tools in Agile

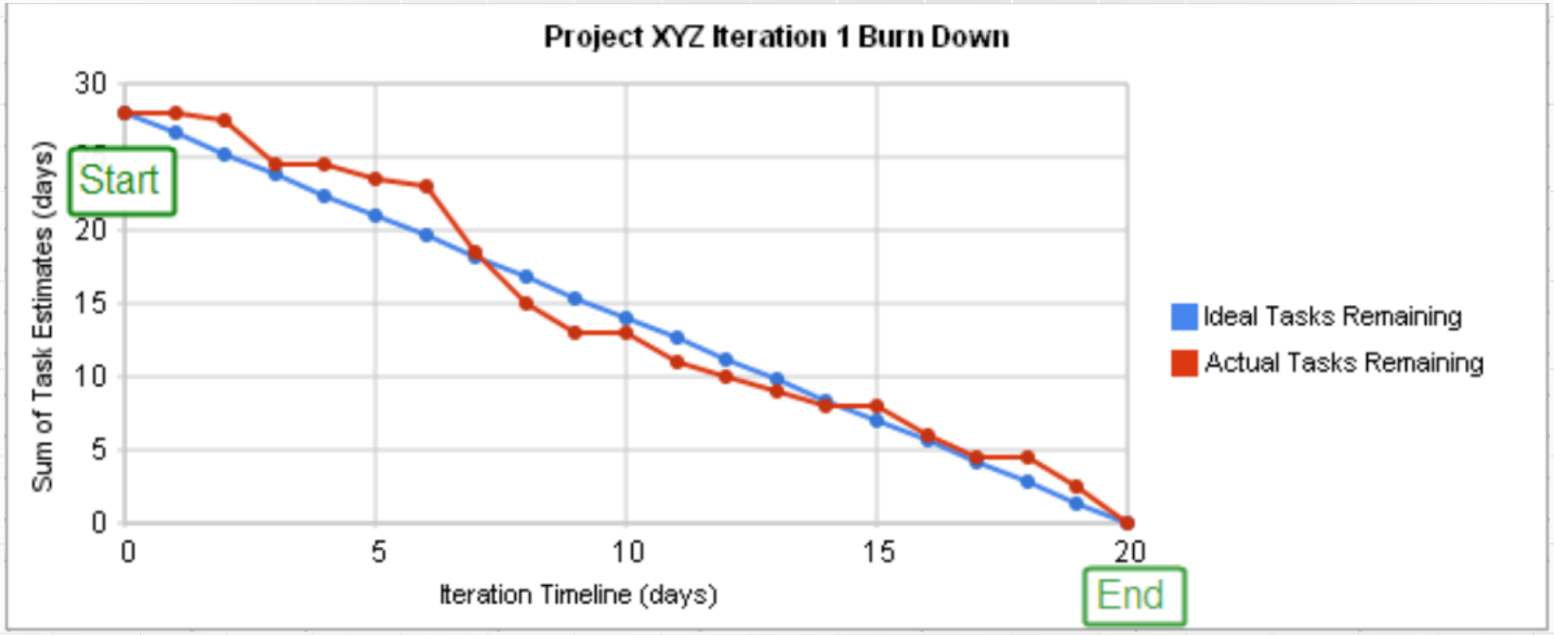
- Both the **backlog** and the **Kanban board** are tools for managing work, but they serve different purposes.
- The backlog defines **what should be done**, while the Kanban board shows **what is being done right now**.

	Backlog	**Kanban Board**
Purpose	Plan and prioritize future work	Visualize and control current work
Content	All upcoming items: stories, bugs, spikes, tasks	Only active items in the current flow
Focus	*"What should we do next?"*	*"What are we doing now?"*
Ownership	Product Owner / team	Whole team
Structure	Ordered list by priority	Columns representing workflow (To Do → In Progress → Done)
Dynamics	Updated during planning	Updated continuously during work

Tasks I

Tasks

- Each story consists of one or more tasks.
- Tasks are owned by individual team members.
- Estimated in hours (typically 4–8 hours).
- Progress is tracked via **task burndown**.
- Tasks can also support refactoring, research, or infrastructure work.



Tasks II

Example:

Story: As a registered user, I can reset my password so that I can regain access. Tasks:

- *Define acceptance criteria — Anna*
- *Design password reset form — Marek*
- *Implement backend API — Ola*
- *Write automated tests — Ola*
- *Update documentation — Kasia*

Spikes

What is a Spike?

A **spike** is a special type of user story used for **research, design, or exploration**. It helps the team reduce uncertainty or risk before implementation.

Characteristics:

- Time-boxed (typically 1–2 days).
- Produces knowledge, not code — outcome may be a prototype, analysis, or decision.
- Helps clarify estimates, technical feasibility, or architectural approach.
- Tracked in the backlog like other stories, but marked as *spike*.

Example:

Spike: Evaluate database options for scalable user session storage. Goal: compare PostgreSQL, Redis, and DynamoDB based on latency and cost.

Agile requirements: Program level

Vision (Program Level)

Definition:

- The **Vision** describes the strategic intent of a product, system, or application.
- It communicates what is to be built, why it matters, and for whom it is intended.
- Replaces traditional documents such as SRS in agile environments.

Purpose:

- Provide a shared understanding of **goals, users, and key benefits**.
- Align teams with business and investment strategy.
- Prevent premature commitment to uncertain or unstable requirements.

Typical questions the Vision answers:

- Why are we building this product?
- What problem does it solve?
- What features and benefits will it deliver?
- Who are the target users or customers?
- What qualities (performance, reliability, scalability) are essential?

Features

Definition

- Services provided by the system that fulfill one or more stakeholder needs.
- Bridge between the **problem domain** (user needs) and the **solution domain** (software requirements).
- Represent high-level capabilities — what the system will do and what benefits it provides.

Characteristics

- Abstract and higher-level than user stories (typically 25–50 per system).
- May be written in user voice form: *As a <user>, I can <action> so that <benefit>*.
- Estimated in story points or t-shirt sizes, refined over iterations.

Roadmap and release planning I

Roadmap

- Describes the intended sequence of major deliverables over time.
- Communicates how vision and features will evolve across upcoming releases.
- Provides visibility and alignment without fixing long-term commitments.
- Continuously updated as priorities and market conditions change.

Roadmap and release planning II

Release Planning

- Defines what will be delivered and when — based on team and program velocity.
- Aligns multiple teams around a shared set of objectives for a release.
- Ensures that dependencies, milestones, and resources are visible and manageable.
- Focuses on delivering value early and adjusting scope rather than shifting deadlines.

Agile requirements: Portfolio level

Investment themes

Definition:

- Investment themes define the strategic areas in which the enterprise decides to invest.
- They guide allocation of funds and resources across systems, products, and services.
- Each theme represents a long-term direction rather than a short-term objective.

Role of investment themes:

- to provide a link between business strategy and portfolio execution.
- to influence portfolio vision and drive the creation of new epics.
- to express investment as a share of total capacity (e.g., 20% for mobile, 15% for security).

Note: Themes are not backlog items. They describe *where* and *how much* the organization invests, not *what order* work is done in.

Epics (Portfolio Level)

Definition:

- An **Epic** is a large, significant initiative that delivers substantial business or architectural value.
- Represents work that is too large to complete within a single release.
- Typically spans multiple teams and several months of development.

Characteristics:

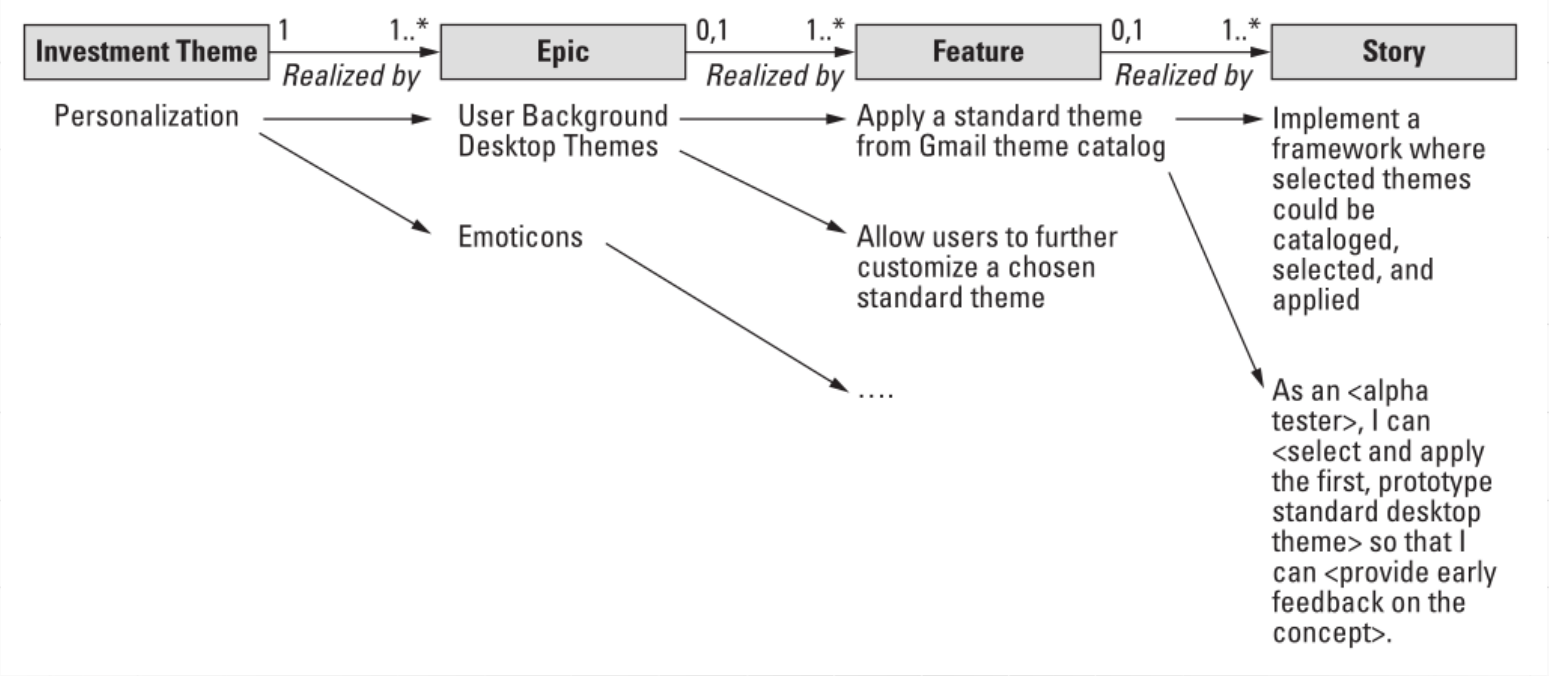
- Originates from strategic objectives or investment themes.
- Decomposed into **Features** and later into **Stories** for implementation.
- Evaluated using a simple business case (value, cost, risk, benefit hypothesis).
- Managed and prioritized at the portfolio or program level.

Outcome of Epic: A validated set of features ready for release planning and team breakdown.

Wroclaw
University
of Science
and Technology

Summary

Example


Investment Theme
1
1..*
Realized by
Epic
0,1
1..*
Realized by
Feature
0,1
1..*
Realized by
Story
Personalization
User Background Desktop Themes
Apply a standard theme from Gmail theme catalog
Implement a framework where selected themes could be cataloged, selected, and applied
Emoticons
Allow users to further customize a chosen standard theme
....
As an <alpha tester>, I can <select and apply the first, prototype standard desktop theme> so that I can <provide early feedback on the concept>.

Summary: Levels of Agile Requirements

Type	Description	Responsibility	Time Frame and Sizing	Testable
Investment Theme	Strategic, long-term initiative defining direction and competitive advantage.	Executives, portfolio management.	12–18+ months, expressed as share of total investment.	No
Epic	Large, market-facing initiative or major capability.	Portfolio management, product management, system architects.	6–12 months, sized in story points or t-shirt sizes.	No
Feature	Deliverable service or functionality providing clear business value.	Product manager, product owner.	1–3 months (within a release), sized in points.	Partially (via acceptance criteria).
Story	Small, atomic item representing user intent and value.	Product owner, development team.	One iteration (few days to weeks), sized in story points.	Yes

Wrocław
University
of Science
and Technology

The metamodel

The metamodel of agile requirements

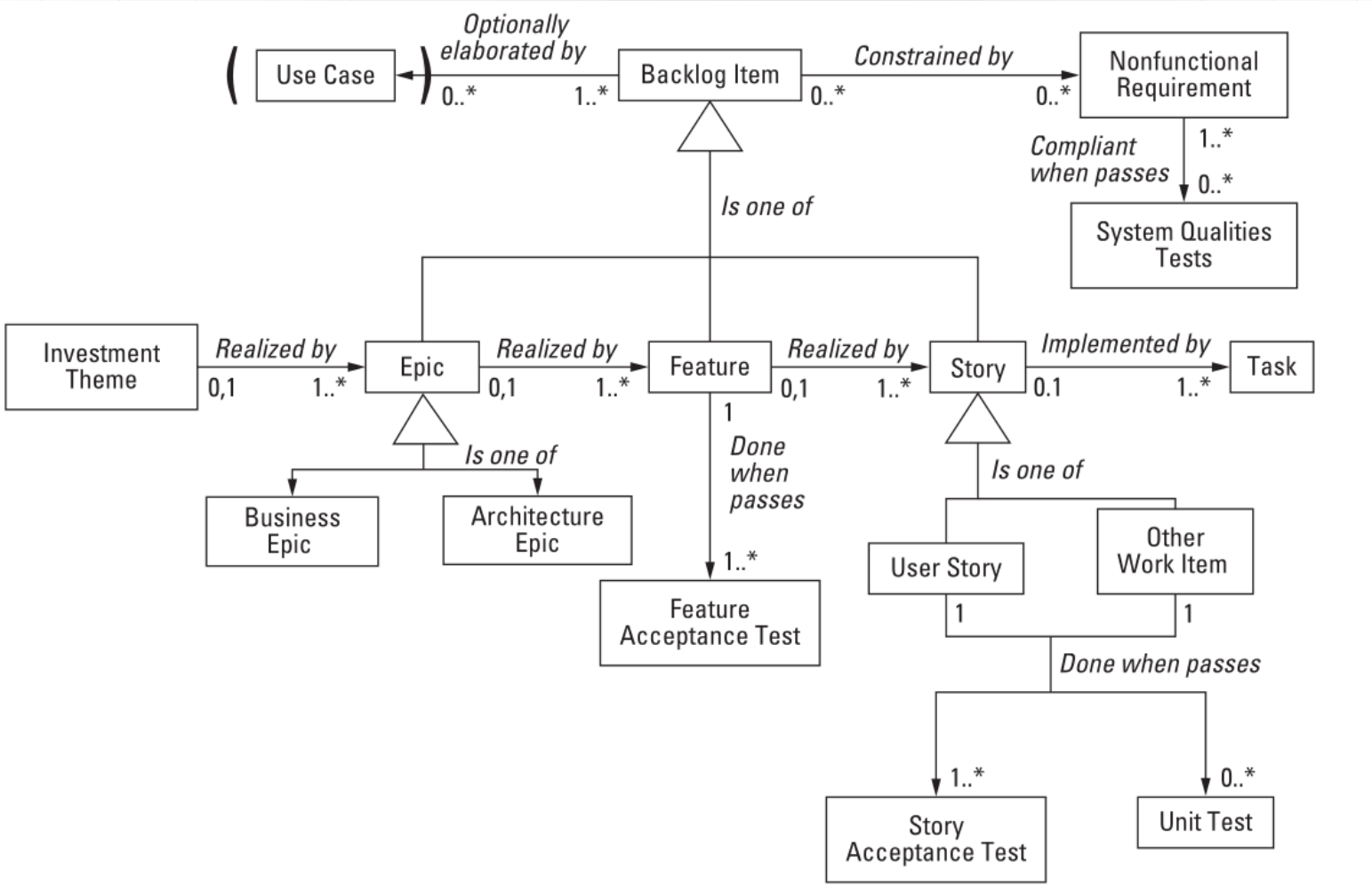


Figure: The metamodel of agile requirements[3]

[3] Leffingwell 2011.

Wrocław University of Science and Technology

It's all for today!